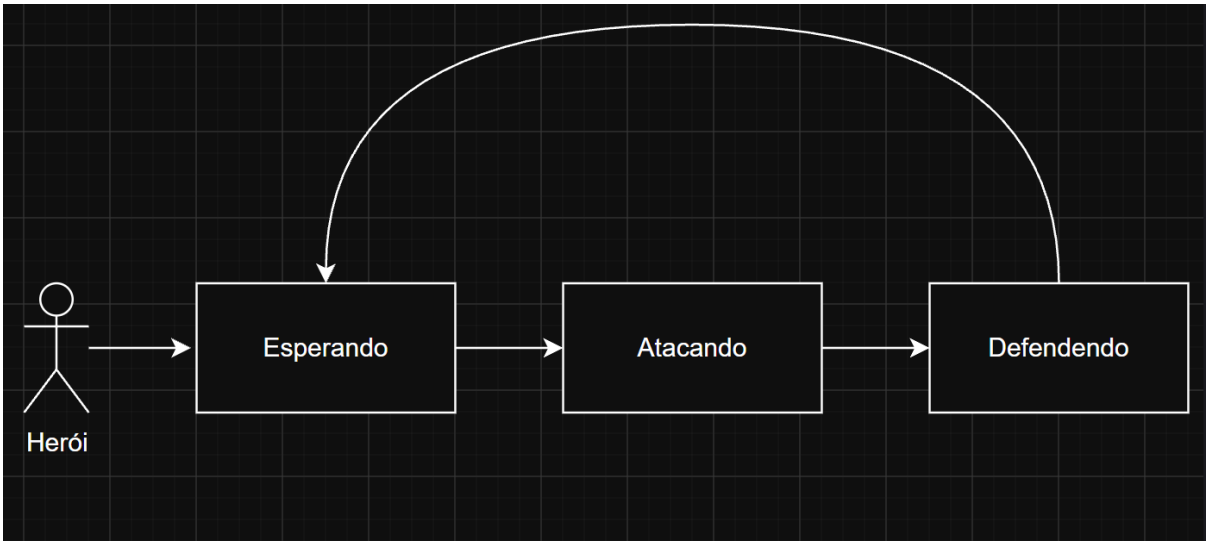
É uma máquina de estado que simula um confronto entre um agente com o papel de Herói e um outro agente com o papel de Monstro, a batalha é de turnos simulando um confronto em um jogo de rpg.

O agente Herói é uma subclasse da classe abstrata "Agente", a classe do herói tem 3 parâmetros, "Nome", "Luta" e "Vida", o herói tem 3 estados, "EstadoHeróiAtacando" que causa um dano aleatório de 0 a 40 ao monstro, "EstadoHeróiDefendendo" que apenas avança o turno, dando a ilusão de defesa e "EstadoHeróiEsperando" que é o estado de transição para os outros estados.

O agente Monstro também é uma subclasse da classe abstrata "Agente", a classe do monstro tem 3 parâmetros, "Nome", "Luta" e "Vida", o monstro tem 2 estados, " EstadoMonstroEsperando " que é o estado de transição para os outros estados e "EstadoMonstroAtacando" que causa um dano de 0 a 40 ao herói.

Diagrama de estados

Herói:



Monstro:

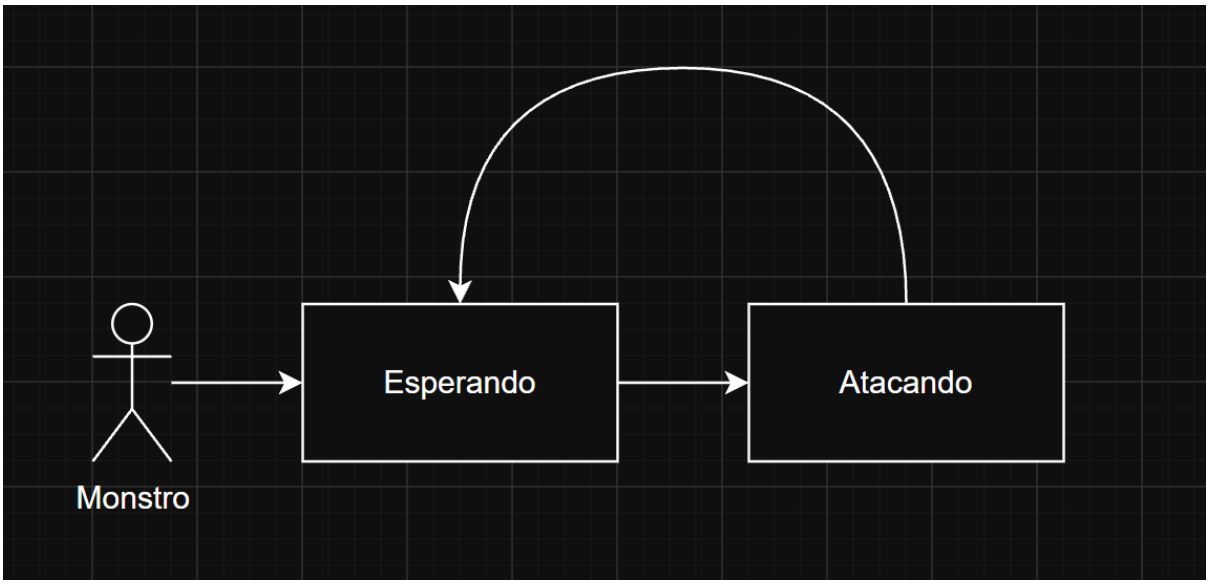


Tabela de regras

Herói:

Herói				
EstadoHeroiEsperando	Padrão	Incrementa `turnosEstado` e loga "esta esperando. Turno no estado: N"	Padrão	`turnosEstado >= 1` → ATACANDO. (ocorre sempre, já no 1º turno)
EstadoHeroiAtacando	Padrão	Incrementa `turnosEstado`. Se a luta acabou, retorna sem fazer nada. Senão loga "está atacando!" e chama `luta.ataque(heroi, monstro, rand 0-40)`, que loga o ataque e aplica `receberDano()` no monstro	Padrão	Se `!fimBatalha()` → DEFENDENDO. Se a luta acabou, permanece no estado
EstadoHeroiDefendendo	Padrão	Incrementa `turnosEstado` e loga "está defendendo." (sem efeito mecânico)	Padrão	`turnosEstado >= 1 && !fimBatalha()` → ESPERANDO

Monstro:

Monstro				
EstadoMonstroEsperando	Padrão	Incrementa `turnosEstado` e loga "Está esperando. Turnos neste Estado: N"	Padrão	`turnosEstado >= 1` → ATACANDO. (sempre, no 1º turno)
EstadoMonstroAtacando	Padrão	Incrementa `turnosEstado`. Se a luta acabou, retorna. Senão loga "está atacando!" e chama `luta.ataque(monstro, heroi, rand 0-40)`que loga o ataque e aplica `receberDano()` no Herói	Padrão	Se `!fimBatalha()` → ESPERANDO. Se a luta acabou, permanece no estado

As variáveis herdadas por ambos:

- Nome
- Vida
- Luta
- estadoAtual

"Nome" é uma string que é usado como identificador nos logs que não muda.

"Vida" é um int que representa a vida do personagem.

"Luta" é uma classe que se refere ao contexto de batalha é por ela que os estados acessam o oponente e disparam ataques.

"estadoAtual" é um estado que começa null que só passa a existir do primeito "mudarExtado()" no "Main()"

Valores específicos de cada agente

Herói:

- Nome: Bartz
- Vida: 100
- Estado Inicial: Esperando
- Dano: 0/40 (aleatório)

Monstro:

- Nome: Catoblepas
- Vida: 80
- Estado Inicial: Esperando
- Dano: 0/40 (aleatório)

Variáveis dos Estados

- Agente
- Nome
- TurnosEstado

"agente" ele é do tipo Agente que é usado para logar o nome acessar a luta e pdeir a troca de estado.

"nome" é uma string que é usada nos logs para mostrar qual estado os agentes estão atualmente.

"turnosEstado" é um int que conta os turnos do estado atual, é zerado em "entrar()" e incrementado em "avancarTurno()" no início de cada "executar()".

Limiares globais

- Limite de turnos
- Turno inicial
- Pausa_MS
- Condição de morte

- Fim da batalha

"Limite de trurnos" é representado por "turno <= 20" no "Gerenciador", usado para encerrar a luta se os dois ainda estiverem vivos.

"Turno inicial" é um int localizado no "Gerenciador" que começa sendo igual a 1, que conta quantas batalhas ocorreram.

"Pausa_MS" é um int localizado no "Log" que tem o valor de 1000, faz com que os logs tenham 1 segundo de delay para aparecer na tela.

"Condição de morte" é uma booleana localizado no "Agente" que faz uma comparação de "vida > 0" para definir se o agente está vivo.

"Fim de batalha" é uma booleana que retorna qual agente que está morto usando "!heroi.estaVivo() || !monstro.estaVivo(); " para o check.

Limiares por estado

Herói:

- "ESPERANDO", acontece quando a condição de “turnosEstado >= 1”, quando as condições forem atendidas, troca o estado para “new EstadoHeroiAtacando”.
- “ATACANDO”, acontece quando o estado “ESPERANDO”, ocorre ao menos uma vez na iteração, dependendo da condição, troca o estado para “DEFENDENDO”
- “DEFENDENDO, acontece quando o estado “ATACANDO” ocorre ao menos uma vez, e caso a luta não termine na última iteração do estado “ATACANDO”(o monstro ainda possui pontos de vida)

Monstro:

- “ESPERANDO”, acontece no início da execução, quando a condição de “turnosEstado >= 1” for atendida. Se verdade, muda o estado para “ATACANDO”
- “ATACANDO”, acontece quando o Monstro passou pelo estado de “ESPERANDO”, caso o Herói ainda tenha pontos de vida, ele retorna ao estado “ESPERANDO”, caso contrário, a luta termina.

Estrutura do código

- **Agente**: É onde guarda o nome, vida, luta e estado atual, também fornece “mudarEstado()” e “executarEstado”, controla a vida com o “receberDano()” e “estaVivo()”.
- **Estado**: É uma interface que define o contrato que todo estado deve cumprir, com “entrar()”, “executar()”, “sair()” e “getNome()”.
- **EstadoBase**: É uma classe abstrata que guarda qual é o agente e o nome do estado, mantém "turnoEstado” e fornece “entrar()” e “sair()” que zeram o contador e registram no log
- **EstadoHeroiAtacando**: Estado de ação do herói, sorteia o dano manda a “Luta” executar o ataque contra o monstro e segue para “EstadoHeroiDefendendo”.
- **EstadoHeroiDefendendo**: Estado de recuperação do herói, registra a defesa no log e retorna a “EstadoHeroiEsperando”.
- **EstadoHeroiEsperando**: Estado de preparação do herói, passa o turno sem agir e segue para “EstadoHeroiAtacando”.
- **EstadoMonstroAtacando**: Estado de ação do monstro, sorteia o dano manda a “Luta” executar o ataque contra o monstro e segue para “EstadoMonstroEsperando”.
- **EstadoMonstroEsperando**: Estado de preparação do monstro passa o turno e segue para “EstadoMonstroAtacando”
- **Gerenciador**: Controla o fluxo da partida, roda o laço de turnos (até 20), executa primeiro o herói e depois o monstro, interrompe quando a luta acaba e anuncia o vencedor.
- **Herói**: Especialização do “Agente” que repassa os dados ao construtor pai, existe como ponto de extensão para atributos próprios, como mana, e dá um tipo próprio para “Luta.getHeroi”.
- **Monstro**: Especialização do “Agente” que repassa os dados ao construtor pai, existe como ponto de extensão para atributos próprios, como mana, e dá um tipo próprio para “Luta.getMonstro”.
- **Luta**: Representa o contexto da batalha, cria o herói e o monstro, dá a cada um acesso ao oponente (getHeroi() e getMonstro()), executa os ataques validando que ninguém morto pode atacar e informa o fim da batalha em “fimBatalha()”, funciona como mediador, pois os estados pedem a ela o ataque em vez de chamar “receberDano()” diretamente.
- **Log**: Cuida da saída de texto. “mostrar()” imprime e espera 1 segundo para dar ritmo à leitura, e "mostrarRapido()” imprime sem pausa.
- **Main**: Ponto de entrada, monta a luta, define o estado inicial de cada agente (ESPERANDO), cria o Gerenciador e o executa.

Resultados esperados

```
[SISTEMA]: Bartz Entrou no estado: ESPERANDO
[SISTEMA]: Catoblepas Entrou no estado: ESPERANDO

========== TURNO 1 ==========
Bartz esta esperando. Turno no estado: 1
[SISTEMA]: Bartz saiu do estado: ESPERANDO
[SISTEMA]: Bartz Entrou no estado: ATACANDO.
Catoblepas Está esperando.  Turnos neste Estado: 1
[SISTEMA]: Catoblepas saiu do estado: ESPERANDO
[SISTEMA]: Catoblepas Entrou no estado: ATACANDO.

========== TURNO 2 ==========
Bartz está atacando!
Bartz ataca Catoblepas!
Catoblepas recebeu 4 de dano! Vida: 76
[SISTEMA]: Bartz saiu do estado: ATACANDO.
[SISTEMA]: Bartz Entrou no estado: DEFENDENDO
Catoblepas está atacando!
Catoblepas ataca Bartz!
Bartz recebeu 12 de dano! Vida: 88
[SISTEMA]: Catoblepas saiu do estado: ATACANDO.
[SISTEMA]: Catoblepas Entrou no estado: ESPERANDO
```

Este é um exemplo do código sendo executado, onde o Sistema mostra os estados atuais dos respectivos atores e quantos turnos ele passou no estado demonstrado. Devido ao fator randômico do contexto de uma luta, a execução pode ser mais duradoura conforme nenhum dos atores atinge a condição de vitória, que é derrubar os pontos de vida do ator oposto a zero.